

安全内存回收:Hazard Pointer 与 RCU

无锁数据结构“何时能安全删除一个节点”问题的两种解法——从一个会崩的栈,到手写的垃圾回收,再到用时序换取零成本读

Robert (Bo) Hou · Robert's Technical Notes · 2026 年 6 月

摘要。无锁并发有两层彼此正交的问题。第一层是顺序与可见性——数据写好了才被读到——由 acquire/release/fence 解决(SPSC、seqlock、MPMC 三份报告的主题)。第二层是内存生命周期:一个节点还有别的线程攥着,就被释放了,导致 use-after-free。第二层是本报告的主题,而它只在需要 new/delete 节点的结构(链表、树)上出现——数组版有界队列靠槽位循环复用天然绕开了它。我们从一个朴素的无锁栈出发,让它在纯 pop-pop 交错下崩溃,由此逼出问题;说明 shared_ptr 为何在读多场景不可用(共享引用计数即共享可写,必然争用);推导 hazard pointer(每线程私有的“正在用”公示 + 删前扫表 + 延迟批量回收),并给出它的完整正确性证明;再推导 RCU,它用一个更弱、更廉价的读侧标记(甚至零显式开销)换取“宽限期”这一时序判据,并给出“宽限期结束即可安全删除”的形式化证明。最后指出二者本质是手写的、局部的垃圾回收,以及为什么避开回收的最优办法是根本不动态分配。

1. 两层正交的问题,以及一个会崩的栈

先把并发的两层问题分清,因为把它们混在一起是理解回收的最大障碍。第一层是顺序/可见性:acquire/release 保证“写好了才读到”,它管的是读到的值对不对。第二层是内存生命周期:一块内存还有人用,就被 free 了,它管的是这块地址还合不合法。acquire/release 对第二层无能为力——它保证不了“指针指向的内存还在不在”。

用一个朴素的无锁栈把第二层的问题逼出来。push 用 CAS 头插;pop 用 CAS 摘下栈顶节点,读出数据,然后 delete 该节点:

```
bool pop(T& out) {
    Node* old_head = head_.load(acquire);           // (1)
    while (old_head != nullptr &&
           !head_.compare_exchange_weak(old_head, old_head->next, // (2)
                                         acquire, relaxed)) { }
    if (old_head == nullptr) return false;           // (3)
    out = old_head->data;                             // (4)
    delete old_head;                                 // (5)
    return true;
}
```

这份代码的内存序是对的(push 发布 data 用 release,pop 成功读 data 用 acquire),但它在纯 pop-pop(两个线程都 pop,连 push 都不需要,也无关 ABA)下会 use-after-free。

崩溃的交错(纯 pop-pop)。 栈 head → A → X → ...。线程 T1、T2 同时 pop。① T1 执行 (1):old_head = A,攥着 A,尚未 CAS。② T2 执行 (1):也拿到 old_head = A。③ T2 跑完 (2)CAS 成功(head 换成 X),再走 (4)(5):读 A 的 data,delete A——A 这块内存被释放。④ T1 醒来执行 (2):CAS 是 compare_exchange(old_head, old_head->next),而进入 CAS 之前必须先求实参 old_head->next = A->next——此刻 A 已被 delete,这一步就是读野内存。

要点被很多人忽略:CAS 本身是原子的,但它救不了这个 bug。原子性保护的是“比较并交换 head”这个动作;而 old_head->next 这个实参是在进入 CAS 之前由普通代码求值的,不在原子范围内。A 被另一个线程 delete,与本次CAS 的原子性正交。原子性解决“不可分割”,解决不了“我拿来解引用的地址已被释放”。

问题的定性。 根因是:“摘下节点就 delete”与“别的线程还攥着该节点”撞车。两个线程共享了同一个 old_head = A,一个把它删了,另一个还在用它。这不是顺序问题(acquire/release 管不了),是生命周期问题:只要还有线程攥着 A,就不能 delete A。而当前代码摘下即删,不问有没有人还攥着——所以崩。安全内存回收(SMR)就是来回答“何时能安全删”的。

2. 为什么 shared_ptr 不行:共享计数即争用

第一个本能是用 `shared_ptr` 的引用计数:“还有人用就别删,计数归零才删。”逻辑上对,但在读极多的热路径上工程上不可用,原因有二。

- 普通 `shared_ptr` 的并发读本身不安全:它内部是两个指针(指对象 + 指控制块),多线程同时读写同一个 `shared_ptr` 变量会撕裂。要安全得用 `atomic<shared_ptr>`(C++20)或旧的 `atomic_load` 那套。
- 致命:引用计数是共享可写状态。每次读者 load 一个节点,都要原子地 ++/-- 挂在控制块上的那个计数——所有读者狂敲同一条 cache line,跨核 ping-pong。读越多争用越狠。而且不少平台 `atomic<shared_ptr>` 内部还退化成锁,`is_lock_free()` 为 false。

核心矛盾。 引用计数把“谁在用”这个信息做成了共享可写状态,而共享可写 = 同步 = 争用。读多的场景,这是灾难。破解之道:把“谁在用”从“一个共享计数”改成“每线程私有的公示”——读者只写自己的、不碰共享,争用消失。这正是 hazard pointer 的出发点。

3. Hazard Pointer:每线程公示 + 删前扫表

hazard pointer 把“我正在用某指针”这条信息,登记到每线程私有的公示槽里(本线程写、其他线程读),从而读侧无争用;删除者删一个节点前,扫一遍所有线程的公示槽,只要没人登记该指针才删。读快、删慢——正合“读多删少”。

3.1 两张表

- Hazard 表:每线程若干固定槽(通常 $O(1)$ 个;一次 pop 只需保护 1 个指针)。读者拿到指针后,把指针值写进自己的槽 = “我在用它”。私有写、全局读。
- Retired
list(待删表):每线程一个私有列表,存“我想删但当时有人还攥着、不敢立刻删”的节点。私有写、私有读。

3.2 流程

- 读者:protect(p) —— 把 p 写进自己的 hazard 槽,再以屏障保证“登记 happens-before 使用”,然后才解引用 p。
- 删除者:节点从结构摘除后,不立刻删,而是 retire(p) —— 把它加进自己的 retired list。攒够阈值 R,触发一次扫描:遍历全部 hazard 槽,retired 里没被任何槽登记的节点真正 delete,还被登记的留到下次。

对用户透明。用户(写无锁栈的人)只碰两个 API:读时 protect、弃用时 retire。什么时候攒够、何时扫描、何时真正 delete,全在库内部,用户看不见。这与 shared_ptr 用户只管构造/析构、引用计数归零时机透明,是同一种抽象——只是这里的“何时删”判定(扫表)更复杂,所以更要藏进库里。用户逻辑结束 $\neq$ 线程结束 $\neq$ 节点被删:节点被移交给库,择日回收。

3.3 一个边界:每线程 hazard 指针必须是有限常数个

每个操作同时保护的指针应是 $O(1)$ 个(pop 通常就 1 个:old_head)。若一个线程试图登记任意多的指针,那是误用 hazard pointer——它的前提就是“每线程同时保护常数个指针”。因此不需要支持无限指针的无锁集合;每线程几个固定原子槽即足,超了说明设计错了。

3.4 正确性证明

hazard pointer 的证明干净——直觉即证明。要证的核心命题是:当一个节点被 delete 时,没有任何线程还攥着它。

命题(安全回收)。设删除者在时刻 T 摘除节点 N(N 结构上不可达),并遵守“delete N 前扫描全部 hazard 槽、确认没有槽 == N 才删”。则 delete N 时,没有任何线程持有指向 N 的可解引用指针。

证明(反证 + 两条 happens-before 边)。假设某线程 T_j 在 N 被 delete 后仍解引用 N。由 hazard pointer 的使用契约, T_j 解引用 N 之前必然先 protect(N)——即把 N 写进自己的 hazard 槽(边①:登记 happens-before 使用,由 protect 之后的屏障保证)。于是在 T_j 这次使用期间,它的槽 == N。

另一方面,删除者在摘除 N(时刻 T)之后才扫描 hazard 表(边②:摘除 happens-before 扫描)。两种情况:(i) 若 T_j 的 protect(N) 发生在删除者扫描之前,则扫描时看到某槽 == N,删除者不会 delete N——与假设矛盾。(ii) 若 T_j 的 protect(N) 发生在扫描之后,那 T_j 是在 N 已摘除(结构不可达)之后才试图获取 N——但摘除后从结构遍历拿不到 N, T_j 无从 protect 一个它够不到的指针,矛盾。

两种情况皆矛盾,故不存在“delete 后仍解引用 N”的线程。■ 证明的命门是两条 happens-before 边:①登记先于使用、②摘除先于扫描;它们由 memory_order(store-load 屏障)保证。这正是 SPSC 那套 happens-before 论证的直接应用——所以说 hazard pointer straightforward。

4. RCU:用时序换取零成本读

hazard pointer 读侧仍非零成本:每次读要写自己的槽(一个 store)+一道屏障。在读极端多(路由表、配置:每秒千万次读、数小时才写一次)的场景,这仍嫌贵。RCU 的野心是让读侧接近零成本——读者几乎什么都不做。

4.1 读侧:每线程私有标记,而非共享计数

若读者“什么都不登记”,删除者怎么知道还有没有人在读?第一个想法是“进临界区库里一个计数 +1、出 -1”——但这恰是 RCU 极力避免的:共享计数即共享可写,所有读者狂敲同一条 cache line,退回引用计数的争用老路。

读侧的正确形态。“我在不在读临界区”这条信息必须每线程私有(本线程写、删除者读),绝不能是共享计数。三个层次,越往下读侧越轻:(1) 共享计数 ± 1 ——有争用,不用;(2) 每线程私有标记进/出——无争用,user-space RCU 常用;(3) 利用天然事件(内核经典 RCU:读临界区内禁止上下文切换,于是“一次上下文切换”天然证明“之前的读临界区已结束”)——读者零显式开销。嵌套用每线程私有的深度计数(depth 归 0 才算“不在”),不是 hashmap、不是共享计数。不同线程的临界区无需一致、无需对应,各读各的即可。

4.2 写侧:复制—替换—延迟回收(read-copy-update)

写者不原地改节点(原地改即并发写,读者会读到半更新),而是复制—修改—原子替换指针:造一个新版本节点,原子地把父指针从旧节点换成新节点。换指针后,新读者看到新节点,但可能还有旧读者攥着旧节点。旧节点最终要 free,但必须先确认无人再读它。

树上的连锁,以及它被什么止住。改一个节点的多字段内容必须复制(多字段更新不原子);但改单个指针字段(父节点指向谁)可以原地原子替换(单指针 store 原子)。所以:若只需改父节点的一个孩子指针,连锁止于父节点(它原地改,不复制);若父节点也需多字段更新,则父节点也复制,连锁沿路径向上传播直到根,最后原子换根指针(path copying,持久化数据结构即如此)。唯一的原子武器是“单指针替换”,多字段更新只能靠复制新版本——两者共同逼出了这套机制。

两个关键前提,读者常在此卡住,这里点清:(前提一)无 parent 指针。若节点带指回父亲的指针,复制一个节点就要改它所有孩子的 parent 指针,连锁炸裂、无法收敛;单向树(只有父—子指针)才能让“复制一条路径”收敛。(前提二)写者从根遍历下来,路径在手。正因为无 parent 指针,写者靠“自己从根走下来记住的路径”找到每一层的父亲去改指针,不需要 parent 指针。(前提三)RCU 要求替换使旧节点从结构上完全不可达——单入边结构(链表、树)天然满足;多入边(图/DAG)需先清掉所有入边,否则 RCU 不直接适用。这些是用户的数据结构更新逻辑,不是 RCU 库的职责;库只提供“读区间标记 + 宽限期回收”。

4.3 回收侧:宽限期与两种删除模式

旧节点从结构摘除后(时刻 T),不能立刻 free——可能有读者在 T 之前进临界区、正攥着它。要等一个宽限期(grace period):从 T 起,每个线程都至少经历一次“临界区之外”的时刻(称 quiescent point / 静默点)。注意时间起点:必须是 T 之后的翻篇,历史上的翻篇不算。令 T' 为最后一个线程翻篇的时刻(宽限期终点),在 T' 而非 T 执行 free。

两种删除模式。同步(synchronize_rcu):摘除后调用它会阻塞,直到宽限期结束,返回后写者自己 free 节点。异步(call_rcu):摘除后传入“节点 + 删除回调”,不阻塞立刻返回;宽限期结束后,由库背后的专门线程/上下文(内核:软中断/kthread;user-space:helper 线程)回调调删。前者写者阻塞,后者写者零等待、回收摊销到后台。多个指针只需共享一个宽限期:凡在 T 前摘除的节点,一次宽限期即可全部覆盖——这是批量回收高效的根源。回调只做“释放这一个旧节点”,与更新阶段的 path-copy 无关:一个造新+摘旧(T 前),一个销毁旧(T' 后),中间隔着宽限期。

4.4 正确性证明

要证的核心命题:宽限期结束(T')后 delete 的节点,没有任何读者还能访问到它。先钉前提。

前提。 (A1) 读者只能在临界区内、从当前可达结构获取(解引用)节点指针。(A2) 读者在某临界区内获取的指针,该临界区 unlock 后不再持有。(W1') 存在摘除时刻 T:T 之后新发起的任何遍历都无法到达 N;但 T 前已在进行的遍历可能仍持有/到达 N。回收规则:在 T'(每个线程在 T 之后都翻篇一次的最晚时刻)执行 free(N)。

证明(简洁版)。取任意 T' 之后的时刻 t,任意读者 R_j。要证 R_j 在 t 不持有 N。R_j 若在 t 持有 N,由 (A1) 它必在某临界区 C 内、且 N 是在 C 内获取的。两种可能:

- C 在 T 之后才开始(lock ≥ T):由 (W1'),C 内的遍历是 T 之后新发起的,到不了 N,与“C 内获取了 N”矛盾。故此种情况不存在——T 之后进的都是新值,拿不到 N。
- C 在 T 之前就开始(lock < T):令 q_j 为 R_j 在 T 之后的第一个 quiescent point(宽限期定义保证存在,且 q_j ≤ T' ≤ t)。q_j 处 R_j 不在任何临界区内,故 C 必在 q_j 之前 unlock;由 (A2),unlock 后 R_j 不再持有 C 内获取的 N。故在 t ≥ T' ≥ q_j 时它已不持有 N,矛盾——T 时攥旧值的,到静默点就松手了。

两种情况皆矛盾,故 T' 之后无读者持有 N,free(N) 安全。■

充分性到底靠什么(“翻篇一次为何够”的确切答案)。不是“翻篇”本身充分,而是两个前提合谋:(W1') 摘除让“翻篇之后再进临界区也捡不回 N”(撑起情况一),(A2) 让“那次攥着 N 的临界区在静默点终止”(撑起情况二)。缺任何一个都不够:没有 (W1'),读者翻篇后又能拿到 N,永远删不掉;没有 (A2),unlock 了还攥着旧指针,静默点形同虚设。“T 之后是新值(W1')、T 时的旧值到静默点就没了(A2)”——两头一夹,T' 之后 N 无人持有。这就是最简洁的正确性核心。

5. 两种解法的统一视角,以及避开回收的办法

hazard pointer 与 RCU
解的是同一个问题——“一块内存何时没人用、可安全回收”——只是判据不同。hazard pointer 是空间判据:登记具体指针,删前扫表看“这个指针在不在”,精确但读侧要写指针。RCU 是时序判据:只标记“在不在临界区”,靠“摘除 + 宽限期”的时序保证安全,读侧几乎零成本,代价是写者要等宽限期(可能较久)。RCU 用时序换掉了精确的指针登记。

它们本质是手写的垃圾回收。	二者都在做	GC
干的事:判断“一个对象还有没有被引用,没有了就回收”。有		GC
的语言(Java)把这件事交给运行时,统一但有停顿;无 GC 的 lock-free C++ 必须自己手写一个局部 GC——hazard 表 / RCU 的宽限期就是“还有没有引用”的判定,retired list / call_rcu 队列就是“延迟批量回收”。回收时机对用户透明(用户只声明 protect/retire 或 call_rcu),这正是 GC 的特征。而这也解释了为什么避开回收的最优办法是根本不动态分配:数组版有界队列槽位循环复用、从不 delete,零回收成本、零延迟不确定性——这是 HFT 偏爱预分配、循环复用、不在热路径 new 的深层原因。链表/树一旦 new/delete,就掉进“何时安全删”的深坑,才需要这套 SMR 机器。		

从一个纯 pop-pop 的 use-after-free,到 hazard pointer 的“公示 + 扫表”并给出 happens-before 证明,到 RCU 的“宽限期”并证明“翻篇一次为何充分”,再到看穿二者与 GC 的同构、以及“避回收即不分配”——安全内存回收这一层至此闭环。每一步都由上一步的破绽逼出,而非从既有机制反推:这正是值得推导、而非背诵的原因。